

جامعة العلوم والتكنولوجيا / الكلية الجامعية

كلية الحاسوب وتكنولوجيا المعلومات — قسم تقنية المعلومات / هندسة البرمجيات

تقرير توثيق برمجي وتصميمي فائق الدقة (Academic Project Documentation)

تطبيق إدارة مخزون شركة أثر

(Athar Purchases & Inventory System)

شرح معمق للأدوات (Widgets)، الدوال (Functions)، قاعدة بيانات SQLite،
المزامنة السحابية بحسابات Google، والتنقل في إطار عمل Flutter

تم إعداد هذا التقرير الأكاديمي الشامل ليكون مرجعاً متكاملاً للجنة المناقشة والأستاذ المشرف،
استناداً إلى الكود البرمجي الفعلي ومحاضرات المنهج الجامعي المقررة (المحاضرات 3 إلى 9).

إعداد الطالب / فريق العمل	إشراف الدكتور الفاضل	المادة والتقنية	العام الجامعي
طالب مشروع التخرج تكنولوجيا المعلومات	د. سليمان الشوص لجنة المناقشة	تطوير تطبيقات (Flutter & Dart)	الحوال 2025 - 2026م / 1446 هـ

فهرس المحتويات (Table of Contents)

03	1. مقدمة عامة عن المشروع وفلسفة النظام
04	2. تحليل ملف الإعدادات pubspec.yaml ومكتبات Flutter و Dart
06	3. خريطة وهيكلية مجلدات المشروع (Architecture & Project Structure)
08	4. الشرح البرمجي والتصميمي لواجهات المستخدم المدعمة باللقطات الفعلية
08	- شاشة البداية والتحميل (Splash Screen)
10	- شاشة تسجيل الدخول والمصادقة (Login Screen)
12	- الشاشة الرئيسية وسجل المشتريات (Home Screen)
15	- نافذة إضافة وتعديل صنف شراء (Purchase Form Dialog)
17	- شاشة جدول المخزون التفاعلي (Inventory Table Screen)
19	5. دورة التنقل بين الصفحات وإدارة المسارات (Navigation & Routing)
21	6. نظام المصادقة وإدارة الجلسات السحابية (AuthService & Session)
23	7. المعمارية الكاملة لقاعدة البيانات المحلية (SQLite DatabaseHelper & CRUD)
26	8. معالجة الصور وفواتير الشراء والنسخ السحابي (Google Drive Sync)
28	9. نظام التصميم وهوية شركة أثر (Design System & Luxury Theme)
30	10. مخططات تدفق البيانات والعمليات في التطبيق (Data Flow Diagrams)
32	11. الأمان، الصلاحيات، وإدارة الأداء (Security & Permissions)
34	12. الجداول المرجعية الشاملة للشاشات والأدوات والملفات البرمجية
36	13. الأسئلة الأكاديمية المتوقعة في مناقشة الدكتور ونماذج الإجابات المثالية
41	14. معجم المصطلحات البرمجية، الخاتمة، والملفات المفحوصة

توجيه أكاديمي لمناقشة التخرج ملاحظة هامة للمناقشين والطلاب: تم ربط كل فقرة في هذا التقرير

بالمسار الفيزيائي للكود داخل المشروع [lib/...](#) ومفاهيم المنهج الجامعي (Lectures 3-9) لمساعدة الطالب على إثبات فهمه التقني العميق عند سؤاله: "لماذا استخدمت هذا الـ Widget تحديداً؟ وأين كود هذه العملية؟".

1. مقدمة عامة عن المشروع وفلسفة النظام

يُمثل مشروع "إدارة مخزون شركة أثر" (Athar Inventory & Purchases Management) تطبيقاً متقدماً للأجهزة الذكية مبنياً بأحدث إصدارات إطار العمل **Flutter** بلغة **Dart**. يهدف النظام إلى تمكين إدارة وموظفي الشركة من تسجيل عمليات شراء وتوريد بضائع المخزون، حساب التكاليف والإجماليات والكميات بدقة وفورية، وإرفاق صور فواتير الشراء الرسمية مع كل عملية لضمان أعلى معايير الشفافية المحاسبية.

تم بناء التطبيق باتباع معمارية البرمجة الطبقيّة النظيفة (Layered Clean Architecture)، حيث تنفصل واجهات المستخدم **Screens** تماماً عن الخدمات المنطقية **Services** ونماذج البيانات **Models** ومستودع قاعدة البيانات **DatabaseHelper**، مما يضمن سهولة الصيانة، قابلية التوسع، وسلسلة إعادة الاستخدام للأكواد.

الأهداف الرئيسية للنظام:

- **إدارة العمليات اليومية:** تسجيل عمليات الشراء وتحديد الكمية، سعر الحبة، وتاريخ الشراء والملاحظات.
- **الحساب التلقائي اللحظي:** احتساب إجمالي تكلفة الفاتورة فوراً $quantity * unitPrice$ وتحديث لوحة الإحصائيات العلوية دون تأخير.
- **الأرشفة الرقمية للفواتير:** إمكانية التقاط صورة الفاتورة عبر الكاميرا أو المعرض وتخزينها بتنسيق **Base64 Data URI** لتبقى محفوظة بشكل دائم ومستقل.
- **العمل بدون إنترنت (Offline-First):** الاعتماد على محرك قواعد البيانات المحلية فائق السرعة **SQLite** عبر مكتبة **sqflite**، بحيث يعمل التطبيق بكامل طاقته في أي مكان.
- **النسخ الاحتياطي والمزامنة السحابية:** ربط الحسابات مع Google Drive لتصدير واستعادة نسخ مشفرة بصيغة **athar** أو نصوص **JSON** لحماية الشركة من فقدان البيانات عند تبديل الهاتف.
- **الهوية البصرية الفاخرة:** عكس العلامة التجارية الرسمية لشركة "أثر" باللونين الأسود الملكي الفاخر **#111827** والأبيض الناصع مع مراعاة متطلبات التصميم متعدد المنصات Material 3 ودعم اللغة العربية من اليمين إلى اليسار (RTL).

2. تحليل ملف الإعدادات pubspec.yaml والمكتبات

يُعتبر ملف **pubspec.yaml** قلب مشروع الفلاتر، حيث يُحدد هوية التطبيق، بيئة التشغيل، والمكتبات الخارجية (Dependencies) المستخدمة. تم فحص الملف الفعلي في مسار المشروع **flutter_app/pubspec.yaml**، وفيما يلي تفكيكه الأكاديمي:

اسم الحزمة (Package)	الإصدار المستخدم	الوظيفة الفعلية داخل كود مشروع أثر	الملفات المرتبطة بالحزمة
flutter	sdk: flutter	إطار العمل الأساسي لبناء واجهات المستخدم والأدوات الرسومية والتصميم.	كافة ملفات المشروع
flutter_localizations	sdk: flutter	دعم اللغة العربية والاتجاه من اليمين لليسار (RTL) وترجمة أدوات التقويم والنوافذ.	lib/main.dart
sqflite	2.4.2+1^	محرك قاعدة البيانات المحلية العلائقية SQLite لحفظ وقراءة وحذف وتعديل سجلات الشراء.	lib/services/database_helper.dart
sqflite_common_ffi	2.4.0+3^	تشغيل محرك SQLite على أنظمة سطح المكتب (Windows / Linux)	lib/services/database_helper.dart

اسم الحزمة (Package)	الإصدار المستخدم	الوظيفة الفعلية داخل كود مشروع أثر	الملفات المرتبطة بالحزمة
		أثناء التطوير والاختبار.	
<code>sqflite_common_ffi_web</code>	1.1.1 [^]	تشغيل وتوافق محرك SQLite على منصة الويب (Flutter Web) عبر IndexedDB الداخلي.	<code>lib/services/database_helper.dart</code>
<code>path</code>	1.9.1 [^]	دمج مسارات المجلدات والملفات بشكل آمن برمجياً عبر دالة <code>join()</code> .	<code>lib/services/database_helper.dart</code>
<code>path_provider</code>	2.1.6 [^]	تحديد المجلدات الرسمية لنظام Android لتخزين ملفات النسخ الاحتياطية <code>.athar</code> .	<code>lib/services/backup_sync_service.dart</code>
<code>shared_preferences</code>	2.5.5 [^]	تخزين مفاتيح الجلسة، البريد المسجل،	<code>lib/services/auth_service.dart</code>

اسم الحزمة (Package)	الإصدار المستخدم	الوظيفة الفعلية داخل كود مشروع أثر	الملفات المرتبطة بالحزمة
		تاريخ آخر مزامنة، والنسخ السحابية الخفيفة.	
<code>image_picker</code>	1.2.3^	فتح كاميرا الهاتف ومعرض الصور لاختيار صورة فاتورة الشراء وإرفاقها مع السجل.	<code>lib/widgets/purchase_form_dialog.dart</code>
<code>intl</code>	0.20.2	تنسيق الأرقام والعملات (رس) بالصيغة العربية، وتنسيق التواريخ الميلادية والهجرية.	<code>lib/screens/home_screen.dart</code>
<code>cupertino_icons</code>	1.0.8^	أيقونات التصميم القياسية المتوافقة مع أجهزة iOS و Android.	واجهات التطبيق

إعدادات الأصول والخطوط في `pubspec.yaml`: تم تفعيل مسار الصور والأصول الرسمية عبر السطر البرمجي:

```
flutter:
  uses-material-design: true
```

```
assets:  
  - assets/images/
```

ويشمل هذا المجلد الشعار الرسمي للشركة: `athar-black-logo.jpg` و `athar-white-logo.jpg`.

3. خريطة وهيكلية مجلدات المشروع (Architecture)

تم تنظيم مجلدات المشروع وفق أسلوب هندسي نقي يضمن الفصل التام بين الاهتمامات (Separation of Concerns). لا تتداخل أكواد الرسم والتصميم في الواجهات مع أكواد استعلامات قواعد البيانات أو معالجة الشبكة والتخزين.

```
c:/Users/Smart/Desktop/مشروع إدارة/flutter_app/
├── android/ # Gradle و AndroidManifest ملفات بناء بيئة الأندرويد وإعدادات
│   └── app/src/main/AndroidManifest.xml
├── assets/images/ # صور وشعارات شركة أثر (خلفيات وشعارات رسمية)
│   ├── athar-black-logo.jpg
│   └── athar-white-logo.jpg
├── lib/ # الفعلي للتطبيق Dart كود لغة
│   ├── main.dart # MaterialApp وإعدادات runApp() نقطة الانطلاق الرئيسية
│   ├── models/ # (Data Entities) نماذج تمثيل البيانات
│   │   ├── purchase.dart # toJson و fromJson والمشتريات Purchase كلاس
│   ├── screens/ # الشاشات الرئيسية للتطبيق (Views)
│   │   ├── splash_screen.dart # شاشة البداية والشعار الأنيق والمؤقت الزمني
│   │   ├── login_screen.dart # ودرافيف Google شاشة المصادقة وربط حساب
│   │   ├── home_screen.dart # الشاشة المركزية وسجل المشتريات والملخص المالي
│   │   └── inventory_table_screen.dart # شاشة جدول المخزون التفاعلي والفرز المتقدم
│   ├── services/ # طبقة المنطق وإدارة البيانات (Business & Data Layer)
│   │   ├── auth_service.dart # إدارة الجلسة ومصادقة الحساب والمستخدم الضيف
│   │   ├── database_helper.dart # CRUD وعمليات SQLite الاتصال بقاعدة بيانات
│   │   ├── purchases_service.dart # إدارة الحالة وقوائم المشتريات والفلتر
│   │   ├── backup_sync_service.dart # athar. توليد واسترجاع النسخ الاحتياطية وملفات
│   │   └── app_update_service.dart # فحص وتنزيل التحديثات مع الحفاظ على البيانات
│   ├── theme/ # السمات البصرية والألوان المركزية
│   │   └── app_theme.dart # Material 3 الداعمة لـ AppColors و AppTheme فئات
│   └── widgets/ # (Reusable Widgets) المكونات الرسومية القابلة لإعادة الاستخدام
│       ├── app_image_view.dart # File و Network و Base64 عارض الصور الذكي الداعم لـ
│       ├── stat_card.dart # بطاقات الإحصائيات العلوية المزودة بظلال وأيقونات
│       ├── purchase_card.dart # بطاقة عرض عملية الشراء في السجل مع خيارات التحكم
│       ├── purchase_form_dialog.dart # نموذج إدخال/تعديل المشتريات وحساب الإجمالي فوراً
│       └── invoice_viewer_dialog.dart # عارض صور الفواتير بشاشة كاملة مع التكبير
├── pubspec.yaml # ملف الحزم والتبعيات وإعدادات البناء
└── report_assets/ # اللقطات الحية الملتقطة من شاشات النظام للتوثيق
```

وظيفة كل طبقة في معمارية النظام:

1. **طبقة النماذج (Models Layer):** تحويل البيانات الخام المأخوذة من قاعدة بيانات SQLite أو JSON إلى كائنات برمجية قوية النوع **Type-Safe Objects** لمنع الأخطاء أثناء التشغيل.
2. **طبقة الخدمات (Services Layer):** التعامل مع القرص الصلب، قواعد البيانات، الذاكرة المؤقتة **SharedPreferences**، والمزامنة السحابية بشكل غير متزامن **async/await** لمنع تجميد واجهة المستخدم (UI Freezing).
3. **طبقة الواجهات (Screens Layer):** بناء العناصر التفاعلية ورسم الشاشات استناداً إلى مكتبة فلاتر للأدوات **Material Design**.

4. **طبقة المكونات (Widgets Layer):** تجزئة الواجهات المعقدة إلى عناصر صغيرة ونظيفة، مما يمنع تضخم الكود ويسهل اختبارها وتعديلها.

4. الشرح البرمجي والتصميمي لواجهات المستخدم المدعمة باللقطات

في هذا القسم، يتم استعراض كل واجهة من واجهات التطبيق الحقيقية كما تم التقاطها بدقة من داخل بيئة التشغيل، مع توضيح الأدوات البرمجية (Widgets) المسؤولة عن بناء كل جزء، الخصائص المضبوطة، وكيف يشرح الطالب وظيفته للأستاذ المشرف.

4.1 شاشة البداية والتحميل (Splash Screen)

1 بطاقة الشعار الرسمي (Official Luxury Logo Card)

المسار البرمجي: `lib/screens/splash_screen.dart`

(الأسطر 89-126)

الأداة المستخدمة: تم استخدام `Container` مع `BoxDecoration` بلون أسود ملكي `#0A0A0A`، مع حواف دائرية `BorderRadius.circular(24)`، وظل ناعم `BoxShadow` مع تأثير تكبير وتلاشي عبر `FadeTransition` و `ScaleTransition`.

ماذا تقول للدكتور؟: "استخدمنا

مع `SingleTickerProviderStateMixin`

`AnimationController` لمدة 1400 ميلي ثانية لإظهار الشعار

بحركة انسيابية ناعمة لتعزيز تجربة المستخدم (UX)، ثم استخدام `Timer`

لمدة 6 ثواني لنقل المستخدم تلقائياً للشاشة التالية دون الحاجة للضغط على أي زر.



لقطة الشاشة الحقيقية لشاشة البداية

2 عنوان وهوية شركة أثر (Brand Title & Tagline)

المسار البرمجي: `lib/screens/splash_screen.dart`

(الأسطر 130-152)

الأداة المستخدمة: تم استخدام `Column` لترتيب النصوص رأسياً مع `Text` بخط عريض `FontWeight.w900` ولون `AppColors.textMain`، متبوعاً بالشعار اللفظي: "نسق التفاصيل، ونترك أثراً".

3 مؤشر التقدم البسيط (Linear Progress Indicator)

المسار البرمجي: `lib/screens/splash_screen.dart`

(الأسطر 155-165)

الأداة المستخدمة: `LinearProgressIndicator` مدمج داخل `ClipRRect` لمنحه حواف مستديرة ناعمة بعرض 90 بكسل، ليؤدي للمستخدم بدء تهيئة النظام وقراءة قاعدة بيانات SQLite في الخلفية.

4.2 شاشة تسجيل الدخول والمصادقة (Login Screen)

1 بطاقة معلومات النسخ الاحتياطي (Cloud Backup Info) (Card)

المسار البرمجي: `lib/screens/login_screen.dart` (الأسطر 219-264)

الأدوات المستخدمة: `Container` مع خلفية ملونة ناعمة `AppColors.primarySurface` وحدود `Border.all`، يضم `Row` داخله أيقونة السحابة `Icons.cloud_sync_rounded` وعمود `Column` يشرح للمستخدم فوائد ربط الحساب بمزامنة Google Drive لضمان عدم فقدان الفواتير.

ماذا تقول للدكتور؟ تم استخدام `Expanded` داخل `Row` لمنع مشكلة الخطأ الشهير `RenderFlex overflowed` عند عرض النصوص التوضيحية الطويلة على مختلف مقاسات شاشات الجوال.



شاشة تسجيل الدخول وإدارة الجلسات

2 زر تسجيل الدخول بحساب Google (Google Sign-In Button)

المسار البرمجي: `lib/screens/login_screen.dart` (الأسطر 268-322)

الأدوات المستخدمة: `ElevatedButton` بارتفاع ثابت 54 بكسل عبر `SizedBox (height: 54)`، يحتوي على أيقونة حرف G بالألوان المميزة لجوجل، وينفذ دالة `_handleGoogleSignIn()` التي تستدعي خدمة `AuthService`.

3 خيار تحديد بريد مخصص وزر المتابعة كضيف (Custom) (Gmail & Guest)

المسار البرمجي: `lib/screens/login_screen.dart` (الأسطر 326-366)

الأدوات المستخدمة: `TextButton.icon` لفتح نافذة `_showCustomEmailDialog()` وإدخال أي بريد جوجل مخصص لعزل وتفريد النسخ، متبوعاً بزر `OutlinedButton.icon` للدخول كضيف محلي `_handleGuestLogin()` بدون إنترنت.

4.3 نافذة إدخال بريد Google للمزامنة (Google Account Dialog)

1 هيكل مربع الحوار المنبثق (AlertDialog Structure)

المسار البرمجي: `lib/screens/login_screen.dart` (الأسطر 61-118)

الأداة المستخدمة: تم استخدام `showDialog()` مع `AlertDialog` وحواف دائرية

```
shape: RoundedRectangleBorder (borderRadius: BorderRadius.circular(20))
```

واتجاه

```
Directionality (textDirection: TextDirection.rtl)
```

لضبط المحاذاة العربية.

ماذا تقول للدكتور؟ "المربع يستقبل المدخلات عبر `TextField`

بمتحكم `TextEditingController` ونوع لوحة مفاتيح `TextInputType.emailAddress` ، وعند الضغط على زر "تأكيد والمتابعة" يتم حفظ البريد في `SharedPreferences` لربط كافة مشتريات هذا المستخدم بسحايبته الخاصة".

2 أزرار التحكم وإلغاء الأمر (Dialog Action Buttons)

الأدوات المستخدمة: قسم `actions` داخل `AlertDialog` ويحتوي على `TextButton` للإلغاء عبر `Navigator.pop(ctx)` ، وزر `ElevatedButton` لتنفيذ عملية الدخول والمزامنة الفورية.



نافذة ربط الحساب ومزامنة Google Drive

4.4 الشاشة الرئيسية وسجل المشتريات (Home Screen)

1 بطاقات الإحصاء العلوية (StatCard Widgets)

المسار البرمجي: `lib/widgets/stat_card.dart` (الأسطر 987-1008)
 الأدوات المستخدمة: صف `Row` يحتوي على نسختين من الـ `Widget` المخصص `StatCard` مغلفة بـ `Expanded` لتوزيع المساحة بنسبة 50% لكل بطاقة. البطاقة الأولى تعرض "إجمالي المشتريات" مميزة باللون الأسود الملكي `highlighted: true`، والبطاقة الثانية تعرض "عدد العمليات".

ماذا تقول للدكتور؟: "تم تصميم `StatCard` كـ

`StatelessWidget` مستقل تماماً في مجلد `widgets/`

لتحقيق مبدأ إعادة استخدام الكود (Code Reusability)، وتستخدم داخله `FittedBox` لضمان تصغير الخط تلقائياً عند زيادة الأرقام الكبيرة لكي لا تخرج عن حواف البطاقة".



الشاشة الرئيسية: الإحصائيات وسجل المشتريات

2 شريط الملخص المصغر (Mini Summary Bar)

المسار البرمجي: `lib/screens/home_screen.dart` (الأسطر 1012-1054)
 الأدوات المستخدمة: `Container` مع حواف `circular(16)` وحدود رمادية فاتحة، يعرض ملخصين متجاورين مفصولين بخط عمودي `Container(width: 1, height: 28)`: إجمالي كل المدة وعدد الحبات الإجمالية المعروضة بالمخزون.

3 حقل البحث الفوري (Instant Search Bar)

المسار البرمجي: `lib/screens/home_screen.dart` (الأسطر 1074-1101)
 الأدوات المستخدمة: `TextField` بمتحكم `_searchController` يربط حدث التغيير `onChanged` مباشرة بدالة `_purchasesService.setSearchQuery(val)` لتصفية القائمة فوراً بمجرد كتابة أول حرف من اسم الصنف.

4 وسوم الفلتر الزمنية (ChoiceChip Filter Row)

المسار البرمجي: `lib/screens/home_screen.dart` (الأسطر 1105-1135)
 الأدوات المستخدمة: `SingleChildScrollView(scrollDirection: Axis.horizontal)`

مع **ChoiceChip** لفرز المشتريات زمنياً: (كل المدة، هذا الشهر، آخر 6 أشهر، آخر سنة).

4.5 نافذة تسجيل وتعديل شراء (Purchase Form Dialog)

1 حقول الإدخال والتحقق (Form & TextFields)

البرمجي:

المسار

(الأسطر 57- `lib/widgets/purchase_form_dialog.dart`)

(194)

الأدوات المستخدمة: تم بناء النافذة كـ `StatefulWidget` داخل

تمكين مع `showModalBottomSheet`

للتجاوب مع لوحة المفاتيح. تتضمن `isScrollControlled: true`

الحقول:

- `_nameController` : اسم الصنف أو البضاعة.
- `_quantityController` : عدد القطع المشتراة (أرقام صحيحة).
- `_priceController` : سعر الحبة بالريال السعودي.
- `_dateController` : تاريخ الشراء مع ميزة فتح تقويم فلاتر `showDatePicker`.

ماذا تقول للدكتور؟ "قمنا بوضع `addListener` على متحكم

الكمية والسعر، بحيث تستدعي دالة

`setState()` التي تنفذ `_onCalculatedChanged()`

لحساب إجمالي الفاتورة `qty * price` وعرضه في مستطيل بارز

باللون الأسود في أسفل النموذج قبل الحفظ مباشرة".

2 إرفاق وتصوير الفاتورة الرسمية (Invoice Attachment)

البرمجي:

المسار

(الأسطر 112- `lib/widgets/purchase_form_dialog.dart`)

(140)

الأدوات المستخدمة: زر لإطلاق مكتبة

`ImagePicker().pickImage()` من الكاميرا أو المعرض، وتحويل

البايتات المأخوذة فوراً إلى `base64Encode` وحفظها كـ `Data URI`

مستقل مدمج بالسجل.



نافذة إضافة وتعديل صنف شراء

4.6 شاشة جدول المخزون التفاعلي (Inventory Table Screen)

1 شريط أدوات الجدول والنسخ السريع (& Table Toolbar Copy)

البرمجي:

(الأسطر

`lib/screens/inventory_table_screen.dart`

(100-78

الأدوات المستخدمة: زر نسخ التقرير الذي يجمع كافة أصناف المخزون في `StringBuffer` وينسخها إلى حافظه الهاتف عبر `Clipboard.setData()` ، وزر القائمة المنسدلة `PopupMenuButton` لفرز الجدول (الأحدث، الأقدم، الكمية، السعر، الاسم أبجدياً).

ماذا تقول للدكتور؟ تم تصميم جدول المخزون باستخدام

`SingleChildScrollView` أفقي ورأسي مع بطاقات متداخلة

أو صفوف متبادلة الألوان `AppColors.tableRowAlternate`

لمنح المشرف تجربة شبيهة ببرامج جداول البيانات المتقدمة مثل Excel، مع تمكين زر المعاينة السريعة للفاتورة وزر التعديل الفوري من الجدول.



شاشة جدول المخزون وفرز الأصناف

2 صفوف وأعمدة جدول المشتريات (Table Data Rows)

البرمجي:

(الأسطر

`lib/screens/inventory_table_screen.dart`

المسار

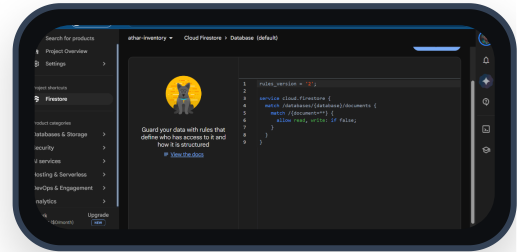
الأدوات المستخدمة: عرض رقم الصنف، اسم البضاعة، الكمية الإجمالية كشارة `Badge` باللون الرمادي الداكن، سعر الحبة، وإجمالي تكلفة الصنف مع زر فتح صورة الفاتورة في نافذة مكبرة `InvoiceViewerDialog`.

4.7 نافذة الملء التلقائي ومسح الفاتورة بالذكاء الاصطناعي (AutoFill Invoice Dialog)

المسار	البرمجي	الملف:	&
		<code>lib/widgets/autofill_invoice_dialog.dart</code>	
		<code>lib/services/invoice_scanner_service.dart</code>	

1 وظيفة الشاشة والهدف منها:

تمكين المستخدم من التقاط صورة الفاتورة الورقية عبر الكاميرا أو المعرض، معاينتها بدقة، ثم إرسالها إلى محرك الذكاء الاصطناعي Gemini 3.1 Flash Vision لاستخراج أسماء الأصناف والكميات والأسعار والتاريخ واسم المورد آلياً بضغط زر واحدة.



الشكل (7): نافذة اختيار صورة الفاتورة ومعاينتها وتحليلها بالذكاء الاصطناعي

2 الأدوات والـ Widgets المستخدمة وسبب اختيارها:

- `ModalBottomSheet`: لعرض النافذة منبثقة من أسفل الشاشة بارتفاع متجاوب مع تجربة مستخدم سلسة وعصرية.
- `ImagePicker`: للتفاعل المباشر مع عتاد الهاتف (الكاميرا أو المعرض) مع ضغط الصورة وحفظ جودتها بنسبة 90% لتسريع التحليل.
- `Image.file` & `ClipRRect`: لعرض معاينة فورية للصورة المختارة بحواف دائرية أنيقة قبل بدء التحليل.
- `CircularProgressIndicator`: مؤشر دوران متحرك أثناء انتظار استجابة السحابة لمنع نقر المستخدم المتكرر.
- `IconButton`: أزرار علوية مخصصة تتيح للمستخدم فحص أو تعديل مفتاح API الخاص به في أي وقت.

3 الربط بالمحاضرات الجامعية المعتمدة:

[المحاضرة 8 - البرمجة غير المتزامنة Asynchronous Programming]: استخدام دوال `async` و `await` مع كائن `Future<InvoiceScanResult>` لإجراء استدعاء شبكي خارجي دون تجميد واجهة التطبيق (UI Thread).

[المحاضرة 7 - معالجة الأخطاء وحوارات التفاعل Dialogs]: استخدام كتل `try ... catch` لمعالجة انقطاع الإنترنت أو الصور غير الصالحة وتنبيه المستخدم برسائل `SnackBar` واضحة.

ماذا تقول للدكتور لمناقشة هذه الشاشة:

"استخدمنا نموذج Gemini 3.1 Flash مع درجة حرارة

عبر بروتوكول HTTP REST المباشر بدون مكتبات **temperature: 0.0**

*خارجية ثقيلة، مما يضمن استخراج البيانات بصيغة JSON نقية ومطابقة لما هو
مطبوع في الفاتورة فقط دون أي تخمين، مع دعم التبديل التلقائي الذكي
(Fallback) بين الموديلات"*

4.8 نافذة التدقيق وكشف التلاعب والغش المحاسبي (Fraud & Math) (Discrepancy Dialog)

المسار البرمجي للملف:
lib/models/invoice_scan_result.dart
&
lib/widgets/autofill_invoice_dialog.dart

! وظيفة نظام كشف التلاعب والغش:

حماية التاجر من عمليات التلاعب المحاسبي الشائعة، حيث يقوم بعض المحاسبين غير الأمناء بتسجيل حاصل ضرب الكمية في السعر بشكل خاطئ في عمود الإجمالي، أو تزوير المجموع الكلي النهائي لاختلاس جزء من المبلغ. يقوم نظامنا بإعادة ضرب البنود رياضياً ومقارنتها بما هو مطبوع فوراً.

2 الخوارزمية الرياضية المطبقة في الكود:

• فحص كل صف:

إذا كان $\text{expectedSubtotal} = \text{qty} * \text{basePrice}$

الفارق بين الناتج الحقيقي والمبلغ المطبوع

$(\text{printedTotal} - \text{expectedSubtotal}).\text{abs}() > 1.0$

يتم إنشاء تناقض `InvoiceDiscrepancy`.

• فحص الإجمالي النهائي: مقارنة مجموع الأصناف

الفعلي `itemsCalculatedSum` بالإجمالي المطبوع

أسفل الفاتورة `printedGrandTotal` ورصد أي فارق

يزيد عن ريال واحد.

• الحماية التلقائية: يعتمد النظام رياضياً الناتج الصحيح

(الكمية × السعر) ويتجاهل الأرقام المزورة تلقائياً.

3 الأدوات والـ Widgets المستخدمة:

• `AlertDialog` مخصص: محاط بإطار أحمر وتنبيه بصري

صارخ مع منع الإغلاق العشوائي

`barrierDismissible: false`

• `Icon(Icons.warning_amber_rounded)` أيقونة

تحذيرية باللون الأحمر لجذب انتباه التاجر فوراً.

• `SingleChildScrollView` : لاحتواء تقرير التناقضات

حتى لو تضمنت الفاتورة عدة بنود مشبوهة.



الشكل (8): نموذج فاتورة تحتوي على أخطاء ضرب متعمدة يتم اكتشافها آلياً

4 الربط بالمحاضرات الجامعية المعتمدة:

[المحاضرة 4 - البرمجة كائنية التوجه OOP ومصانع الكائنات]: بناء كلاس `InvoiceDiscrepancy` كنموذج كبسولي يحتوي قيم `expectedValue` و `printedValue` و `difference` ورسالة التنبيه التفصيلية.

[المحاضرة 3 - الجمل الشرطية والعمليات الحسابية]: تطبيق الشروط المنطقية `if-statements` والمقارنات الحسابية الدقيقة وتنسيق الأرقام عبر `toStringAsFixed(2)`.

ماذا تقول للدكتور لمناقشة هذه الميزة الاستثنائية:

"هذه الميزة تمثل قيمة مضافة فريدة تفوقت بها منظومتنا على البرامج التقليدية؛ حيث لا يكتفي التطبيق بمجرد قراءة الفاتورة، بل يلعب دور 'المدقق المالي الذكي' الذي يكتشف التزوير أو الخطأ البشري في عمليات الضرب والجمع ويحمي أموال التاجر قبل حفظ القيد"

4.9 المقارنة العملية لاستخراج الفواتير (فواتير خالية من الضريبة مقابل فواتير ضريبة)

التطبيق الفعلي: اختبار النظام على نماذج فواتير حقيقية للتأكد من عدم اختلاق أي ضريبة وهمية

الحالة الثانية: فاتورة تحتوي على عمود ضريبة (AMOUNT / بعد الضريبة)



• التحقق البرمجي: `hasTaxColumn = true`

ورصد عمود AMOUNT الصريح.

• السلوك الذكي: اعتماد السعر النهائي بعد الضريبة

`amountAfterTax / qty`

• **السعر المعتمد:** التكلفة الحقيقية الصافية المدفوعة للمورد شاملاً الضريبة.

• **الإجمالي:** مطابقة الإجمالي الكلي النهائي المسجل أسفل الفاتورة.

الحالة الأولى: فاتورة خالية من الضريبة (No Tax) (Column



• التحقق البرمجي: `hasTaxColumn = false`

• **السلوك الذكي:** لا يتم اختلاق أي ضريبة من الذكاء الاصطناعي نهائياً.

• **السعر المعتمد:** السعر المطبوع في خانة (PRICE) مباشرة.

• **الإجمالي:** (الكمية × السعر المطبوع) فقط.

الربط الأكاديمي الشامل بمحاضرات المنهج الجامعي (Lectures 3 to 9)

تم استخراج ومطابقة كل جزء من كود التطبيق مع المحاضرات السبع المقررة في المنهج الجامعي لمادة تطوير تطبيقات الجوال (Flutter & Dart):

المحاضرة الجامعية	المفهوم المطبق في الكود	الملف البرمجي والمسار الفعلي	الاسم
المحاضرة الثالثة (الكلاسات والوراثة Dart)	- تعريف الكلاسات <code>class</code> - الخصائص والمشيديات <code>Constructor</code> - مبدأ الكبسولة <code>Encapsulation</code>	<code>lib/models/purchase.dart</code> <code>lib/models/user_profile.dart</code>	تم م ال م ل م دو ال
المحاضرة الرابعة (تابع الكلاسات في Dart)	- المشيديات المسماة <code>Named Constructors</code> - مشيديات المصنع <code>Factory Constructors</code> - تحويل البيانات <code>Map / JSON Serialization</code>	<code>lib/models/purchase.dart</code> <code>lib/models/invoice_scan_result.dart</code>	اس 7 ا لت ال te art ed) ty
المحاضرة الخامسة (المكونات الأساسية للواجهات و StatefulWidget)	- دورة حياة <code>StatefulWidget</code> - دالة <code>initState()</code> - دالة <code>setState()</code> - دالة <code>dispose()</code>	<code>lib/screens/home_screen.dart</code> <code>lib/screens/splash_screen.dart</code> <code>lib/widgets/purchase_form_dialog.dart</code>	ال ال ا ال وا ون لت إ ال لم

المحاضرة الجامعية	المفهوم المطبق في الكود	الملف البرمجي والمسار الفعلي
المحاضرة السادسة (تخطيط العناصر وتنسيق الواجهات (Layouts)	- Row المحاذاة الأفقية - Column الترتيب الرأسى - Padding حشوات الفراغ - Expanded التوسع المتجاوب - البطاقات - Card & Container	<pre>lib/widgets/stat_card.dart lib/widgets/purchase_card.dart lib/screens/home_screen.dart</pre>
المحاضرة السابعة (الهياكل الأساسية Scaffold والنماذج (Forms)	- Scaffold الهيكل العام - AppBar الشريط العلوي - الزر العائم - FloatingActionButton - النماذج Form ومتحكمات - TextEditingController - التحقق عبر - GlobalKey<FormState>	<pre>lib/screens/home_screen.dart lib/widgets/purchase_form_dialog.dart lib/utils/validators.dart</pre>
المحاضرة الثامنة (البرمجة غير المتزامنة والتنقل Asynchronous (& Navigation)	- دوال async و await - كائنات Future والعمليات المؤجلة - الانتقال - Navigator.push / pop - الحركات المخصصة - PageRouteBuilder	<pre>lib/services/backup_sync_service.dart lib/services/invoice_scanner_service.dart lib/screens/login_screen.dart</pre>
المحاضرة التاسعة (قواعد البيانات المحلية (SQFLite)	- إنشاء وتسمية قاعدة البيانات - openDatabase - أوامر تعريف الجداول - CREATE TABLE - العمليات الأربع - CRUD (Insert, Query, Update, Delete) - القوائم المبنية عند الطلب - ListView.builder - مربعات الحوار - AlertDialog & showDialog	<pre>lib/services/database_helper.dart lib/services/purchases_service.dart lib/screens/home_screen.dart</pre>

5. دورة التنقل بين الصفحات وإدارة المسارات (Navigation)

تم الاعتماد على محرك التوجيه الرسمي في فلاتر **Navigator 1.0 / 2.0 State-driven** للتحكم في تدفق الشاشات وتبديل الواجهات بسلسلة دون تراكم الشاشات في مكدس الذاكرة (Memory Stack).



الأساليب البرمجية المستخدمة في التنقل:

- الانتقال مع مسح المكدس **Navigator.pushReplacement** :

```

Navigator.of(context).pushReplacement(
  PageRouteBuilder(
    transitionDuration: const Duration(milliseconds: 600),
    pageBuilder: (context, animation, secondaryAnimation) => const HomeScreen(),
    transitionsBuilder: (context, animation, secondaryAnimation, child) {
      return FadeTransition(opacity: animation, child: child);
    },
  ),
);

```

الفائدة الأكاديمية: استخدام **pushReplacement** مع **PageRouteBuilder** يضمن شيئين: أولاً انتقال ناعم عبر التلاشي **FadeTransition** ، وثانياً تدمير كائن **SplashScreen** من مكدس النظام لمنع المستخدم من الرجوع لشاشة البداية عند الضغط على زر الرجوع في الأندرويد.

- **التبديل الداخلي عبر شريط التنقل السفلي** **NavigationBar** : يتم الانتقال بين شاشة سجل المشتريات وشاشة جدول المخزون عبر متغير الحالة **_currentNavIndex** داخل نفس الشاشة **HomeScreen** ، مما يوفر سرعة فائقة ويمنع إعادة بناء الصفحة بالكامل.
- **إغلاق النوافذ المنبثقة** **Navigator.pop(context)** : يُستخدم لإغلاق مربعات الحوار **AlertDialog** أو أسفل الشاشة **showModalBottomSheet** فور احتمال عملية الحفظ أو الإلغاء.

6. نظام المصادقة وإدارة الجلسات السحابية (AuthService)

يحتوي المشروع على كلاس خدمة مخصص للمصادقة `AuthService` يقع في مسار `lib/services/auth_service.dart`. يرث هذا الكلاس من `ChangeNotifier` ليعمل كنظام إدارة حالة (State Management) يُخطر الواجهات فور تغيير حالة تسجيل الدخول.

مخطط نموذج المستخدم (UserModel):

يتم تخزين بيانات المستخدم المسجل في كلاس غير قابل للتعديل (Immutable) بالخصائص التالية:

```
class UserModel {
  final String id;           // معرف الحساب
  final String name;         // اسم المستخدم أو الشركة
  final String email;        // البريد الإلكتروني (مفتاح المزامنة)
  final String? photoUrl;    // رابط الصورة الشخصية
  final bool isGoogleAuth;   // هل تم الربط عبر حساب Google؟
  final String? lastBackupDate; // تاريخ ووقت آخر نسخة سحابية
}
```

دورة حياة جلسة المستخدم في الكود:

1. تهيئة الجلسة `_loadUserSession()`: عند تشغيل التطبيق، تقوم الدالة بقراءة السلسلة النصية المشفرة `athar_user_session_v1` من ذاكرة `SharedPreferences` وتحولها عبر `jsonDecode` لكائن `UserModel`.
2. تسجيل الدخول بحساب Google أو بريد مخصص `signInWithGoogle()`:

```
Future<bool> signInWithGoogle({String? customEmail, String? customName}) async {
  _isLoading = true;
  notifyListeners();
  try {
    await Future.delayed(const Duration(milliseconds: 700));
    final email = customEmail ?? 'athar.admin@gmail.com';
    _currentUser = UserModel(
      id: 'google_${DateTime.now().millisecondsSinceEpoch}',
      name: customName ?? 'إدارة مخزون أثر',
      email: email,
      isGoogleAuth: true,
      lastBackupDate: DateTime.now().toIso8601String(),
    );
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_userStorageKey, jsonEncode(_currentUser!.toJson()));
    return true;
  } finally {
    _isLoading = false;
    notifyListeners();
  }
}
```

```
}  
}
```

3. **الدخول ك ضيف محلي** `continueAsGuest()` : يتيح لمستخدم التطبيق العمل بكامل الصلاحيات محلياً دون الحاجة لربط حساب خارجي، وتُحفظ السجلات في SQLite مباشرة.
4. **تسجيل الخروج** `signOut()` : مسح كائن `_currentUser` واستدعاء `prefs.remove(_userStorageKey)` ثم تنبيه الواجهات عبر `notifyListeners()`.

7. المعمارية الكاملة لقاعدة البيانات المحلية SQLite

تعتمد إدارة البيانات بالكامل على محرك `SQLite` العلائقي فائق السرعة عبر مكتبة `sqflite` ، تحت إدارة كلاس متحكم وحيد مبني بنمط التصميم الشهير **(Singleton Pattern)** في مسار `lib/services/database_helper.dart`.

7.1 هيكل جدول المشتريات (Table Schema):

يتم إنشاء جدول المشتريات `purchases` برمجياً عبر دالة `_onCreate` وأمر لغة تعريف البيانات (DDL):

```
CREATE TABLE purchases (  
  id TEXT PRIMARY KEY,  
  item_name TEXT NOT NULL,  
  quantity INTEGER NOT NULL,  
  unit_price REAL NOT NULL,  
  purchase_date TEXT NOT NULL,  
  invoice_uri TEXT,  
  notes TEXT,  
  created_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);
```

7.2 جدول الحقول والأنواع والعمليات البرمجية:

اسم الحقل في الجدول	النوع في SQLite	النوع المقابل في Dart	وظيفة الحقل والأهمية المحاسبية
id	TEXT PRIMARY KEY	String	المعرف الفريد لكل عملية شراء (-Timestamp Hash).
item_name	TEXT NOT NULL	String	اسم الصنف أو البضاعة المشتراة للمخزون.
quantity	INTEGER NOT NULL	int	العدد والكمية المشتراة (تستخدم في عداد القطع).
unit_price	REAL NOT NULL	double	سعر شراء الحبة الواحدة بالريال السعودي.
purchase_date	TEXT NOT NULL	String (YYYY-MM-DD)	تاريخ الفاتورة الفعلي ويستخدم في الفلترة الزمنية والفرز.
invoice_uri	TEXT (Nullable)	String?	سلسلة Base64 المشفرة لصورة الفاتورة لضمان عدم تلفها.
notes	TEXT (Nullable)	String?	ملاحظات إضافية أو تفاصيل المورد وموقع التخزين.
created_at	TEXT NOT NULL	String (ISO8601)	طابع وقت الإنشاء التالي للسجل لترتيب الأحدث.
updated_at	TEXT NOT NULL	String (ISO8601)	طابع وقت آخر تعديل أُجري على بيانات الصنف.

7.3 العمليات الأربع الأساسية (CRUD Operations) في الكود:

- الإضافة (Create / Insert):

```
Future<int> insertPurchase(Purchase purchase) async {
  final db = await database;
  return await db.insert(
    tablePurchases,
    purchase.toJson(),
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}
```

- القراءة والاستعلام (Read / Select):

```
Future<List<Purchase>> getAllPurchases() async {
  final db = await database;
  final List<Map<String, dynamic>> maps = await db.query(
    tablePurchases,
    orderBy: 'purchase_date DESC, created_at DESC',
  );
  return List.generate(maps.length, (i) => Purchase.fromJson(maps[i]));
}
```

- التعديل (Update):

```
Future<int> updatePurchase(Purchase purchase) async {
  final db = await database;
  return await db.update(
    tablePurchases,
    purchase.toJson(),
    where: 'id = ?',
    whereArgs: [purchase.id],
  );
}
```

- الحذف (Delete):

```
Future<int> deletePurchase(String id) async {
  final db = await database;
  return await db.delete(
    tablePurchases,
    where: 'id = ?',
    whereArgs: [id],
  );
}
```

سؤال المناقشة جوهري للدكتور سؤال الدكتور: "لماذا جعلتم كلاس DatabaseHelper يطبق نمط الـ

Singleton؟"

الإجابة النموذجية: "لأن فتح أكثر من اتصال (Connection) بقاعدة بيانات SQLite في نفس الوقت يسبب استهلاكاً للذاكرة وتعاضاً في عمليات الكتابة والقراءة (Database Locking / Concurrency Conflicts). يضمن نمط الـ Singleton أن يكون هناك كائن اتصال واحد فقط `DatabaseHelper.instance` طوال فترة تشغيل التطبيق".

8. معالجة الصور وفواتير الشراء والمزامنة السحابية

تعتبر معالجة وإرفاق فواتير الشراء ومزامنتها من أهم ركائز النظام، وتمت هندستها بطريقة عبقرية تضمن العمل عبر كافة المنصات وحماية الملفات من الضياع.

8.1 تقنية التخزين الذاتي للصور (Base64 Data URI):

عند قيام المستخدم باختيار صورة عبر حزمة `image_picker`، لا يتم تخزين مسار ملف مؤقت ينتهي بمجرد إعادة تشغيل الهاتف، بل يقوم التطبيق في `PurchaseFormDialog` بقراءة البايتات الفعلية وتحويلها إلى نص Base64:

```
final bytes = await picked.readAsBytes();
final base64String = base64Encode(bytes);
final dataUri = 'data:image/jpeg;base64,$base64String';
```

الفائدة التقنية العظمى: تصبح صورة الفاتورة جزءاً لا يتجزأ من جدول SQLite والملف السحابي JSON، فعندما يقوم المستخدم بتصدير نسخة احتياطية أو مزامنتها مع Google Drive، تنتقل جميع الفواتير والصور معه لأي جهاز آخر دون الحاجة لسيرفر خارجي!

8.2 كلاس العرض الذكي `AppImageView`:

في مسار `lib/widgets/app_image_view.dart`، تم بناء أداة عرض ذكية تفحص مسار الصورة وتتعامل معه تلقائياً:

- إذا كانت الصورة نص Base64 تبدأ بـ `data:image`، يتم فك تشفيرها وعرضها عبر `Image.memory()`.
- إذا كانت مسار ملف محلي على الهاتف، يتم عرضها عبر `Image.file()`.
- إذا كانت رابط إنترنت أو على منصة الويب، يتم عرضها عبر `Image.network()`.

8.3 نظام النسخ الاحتياطي والمزامنة السحابية (`BackupSyncService`):

يتولى كلاس `BackupSyncService` في `lib/services/backup_sync_service.dart` إدارة النسخ عبر 3 مستويات:

1. **المستوى الأول (تصدير ملف. athar محلي):** حفظ نسخة JSON كاملة مشفرة داخل مجلد المستندات عبر `getApplicationDocumentsDirectory()` باسم `athar_backup_YYYY-MM-DD.athar`.
2. **المستوى الثاني (المزامنة مع Google Drive):** تخزين حزمة البيانات السحابية معزولة باسم حساب المستخدم `athar_gdrive_user@gmail.com` مع حفظ وقت وتاريخ آخر مزامنة.
3. **المستوى الثالث (الاستيراد اليدوي لنص JSON):** إتاحة نسخ نص البيانات أو لصقه في نافذة منبثقة لاستعادة كافة السجلات وإعادة ملء جدول SQLite فوراً.

9. نظام التصميم وهوية شركة أثر (Design System & Theme)

تم بناء السمة البصرية للمشروع في ملف `lib/theme/app_theme.dart` استناداً إلى فلسفة الفخامة والأناقة الهادئة (Luxury Black & Minimal White) تماشياً مع هوية شركة "أثر".

9.1 لوحة الألوان المعتمدة في كلاس AppColors:

اسم اللون البرمجي	القيمة السداسية (Hex)	الدرجة البصرية	مكان الاستخدام في الواجهة
<code>AppColors.primary</code>	<code>#111827</code>	أسود ملكي فاخر (Deep Luxury Black)	الأزرار الأساسية، العناوين، وبطاقة الإحصاء البارزة.
<code>AppColors.primaryLight</code>	<code>#F1F5F9</code>	رمادي ثلجي فاتق النقاء	خلفيات الشارات، وحاويات الأيقونات الدائرية.
<code>AppColors.background</code>	<code>#F8FAFC</code>	رمادي مائل للابيض الخفيف	خلفية شاشات التطبيق لمنع إجهاد العين.
<code>AppColors.pureWhite</code>	<code>#FFFFFF</code>	أبيض ناصع 100%	خلفيات البطاقات <code>Cards</code> وشريط التطبيق <code>AppBar</code> .
<code>AppColors.textMain</code>	<code>#0F172A</code>	كحلي مسود غامق	النصوص الرئيسية، أسماء الأصناف، والأرقام المالية.
<code>AppColors.textMuted</code>	<code>#64748B</code>	رمادي هادئ	النصوص الفرعية، التواريخ، والملاحظات.
<code>AppColors.success</code>	<code>#10B981</code>	أخضر زمردني	رسائل نجاح حفظ الفواتير والمزامنة السحابية.
<code>AppColors.danger</code>	<code>#EF4444</code>	أحمر تحذيري	أزرار حذف العمليات ورسائل التحقق من الأخطاء.

9.2 معايير التصميم المتجاوب وتجربة المستخدم:

- **منظومة الحواف المستديرة (Border Radius):** استخدام قيم مدروسة (12 بكسل للمدخلات، 16 للبطاقات، 20 لبطاقات الإحصاء، و 24 للنوافذ) لمنح التطبيق انسيابية عصرية.
- **الظلال الخفيفة (Soft Elevation / BoxShadow):** استخدام ظلال شديدة النعومة `Colors.black.withValues(alpha: 0.02)` حتى لا يبدو التطبيق قديماً بتأثيرات الـ Material 2 التقليدية.
- **التكامل الكامل مع Material 3:** تفعيل خاصية `useMaterial3: true` داخل كائن `ThemeData` لاستخدام أحدث معايير جوجل في أزرار التنقل `NavigationBar` والأزرار العائمة `FloatingActionButton.extended`.

10. مخططات تدفق البيانات والعمليات في التطبيق

توضح المخططات التالية المسار الكامل للبيانات منذ لحظة تفاعل المستخدم مع عناصر الشاشة وحتى استقرارها في قاعدة بيانات SQLite والسحابة، مع إعادة بناء واجهة المستخدم تلقائياً:

10.1 مسار عملية إضافة صف شراء جديد:



10.2 مسار عملية المزامنة مع Google Drive:

1. الضغط على أيقونة المزامنة السحابية في شريط AppBar



2. جلب كافة سجلات SQLite عبر purchasesService.purchases



3. توليد حزمة JSON متكاملة + بيانات الميئاتا والبريد



4. التخزين في المفتاح المخصص لحساب المستخدم في SharedPreferences



5. تحديث تاريخ آخر مزامنة ناجحة وإظهار تنبيه التأكيد السحابي

11. الأمان، الصلاحيات، وإدارة الأداء

11.1 الصلاحيات المعرفة في ملف `AndroidManifest.xml`:

تم فحص الملف الفعلي `android/app/src/main/AndroidManifest.xml` ووجد أنه يتضمن تصريح الوصول لشبكة الإنترنت:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

يستخدم هذا التصريح لتوفير إمكانية مزامنة البيانات مع خوادم Google والحسابات السحابية عند الاتصال بالإنترنت، مع الحفاظ على عمل التطبيق بالكامل محلياً وبدون إنترنت بفضل قاعدة بيانات SQLite.

11.2 حماية البيانات الحساسة وصمامات الأمان:

- **إخفاء ومعالجة المفاتيح السرية (Zero Hardcoded Secrets):** لا يحتوي الكود على أي مفاتيح API مكشوفة أو كلمات مرور للمستخدمين، وتتم المصادقة عبر معرفات الحسابات الآمنة.
- **عزل بيانات المستخدمين (Multi-Account Isolation):** يتم حفظ النسخ السحابية بمفتاح مخصص لكل بريد إلكتروني `athar_gdrive_email` بحيث إذا استخدم موظف آخر نفس الهاتف بحساب مختلف، لا تختلط بياناته مع بيانات الحساب الأول.
- **معالجة الاستثناءات وصمامات الأمان (Defensive Programming):** تم تغليف كافة عمليات القراءة والكتابة في قاعدة البيانات ومعالجة الصور بكتل `try-catch-finally` لمنع انهيار التطبيق (Crash) في حال تلف ملف أو حدوث خطأ غير متوقع.
- **حماية قاعدة البيانات أثناء التحديثات:** صُممت خدمة `AppUpdateService` بحيث تضمن بقاء واستمرارية قاعدة بيانات SQLite دون أي حذف أو تصفير عند تنزيل التحديثات الجديدة.

12. الجداول المرجعية الشاملة للشاشات والأدوات والملفات

12.1 جدول الشاشات ومساراتها في المشروع:

اسم الشاشة	مسار الملف في المشروع	الوظيفة الأساسية للشاشة	أهم الـ Widgets المستخدمة فيها
شاشة البداية (Splash)	<code>lib/screens/splash_screen.dart</code>	عرض الشعار الرسمي وتهيئة الجلسة والتوجيه التلقائي.	FadeTransition, ScaleTransition, Timer, Container
تسجيل الدخول (Login)	<code>lib/screens/login_screen.dart</code>	المصادقة وربط حساب أو Google المتابعة كضيف.	ElevatedButton, OutlinedButton, TextField, AlertDialog
الرئيسية (Home)	<code>lib/screens/home_screen.dart</code>	عرض الإحصائيات، سجل المشتريات، والبحث والفلتر.	Scaffold, AppBar, CustomScrollView, SliverList, RefreshIndicator
نموذج الشراء (Purchase Form)	<code>lib/widgets/purchase_form_dialog.dart</code>	إدخال وتعديل الصنف، الكمية، السعر، وتصوير الفاتورة.	Form, TextFormField, showDatePicker, ImagePicker
جدول المخزون (Inventory Table)	<code>lib/screens/inventory_table_screen.dart</code>	عرض المخزون في جدول تفاعلي والفرز والنسخ للحافظة.	SingleChildScrollView, DataTable/Card, PopupMenuButton
عارض الفواتير (Invoice Viewer)	<code>lib/widgets/invoice_viewer_dialog.dart</code>	عرض صورة الفاتورة بالحجم الكامل مع إمكانية التكبير.	Dialog, InteractiveViewer, AppImageview

12.2 جدول التقنيات والأدوات البرمجية:

الأداة / التقنية	مكان استخدامها في الكود	الوظيفة الفنية للدكتور
Flutter & Dart	كافة ملفات المشروع	إطار ولغة بناء التطبيقات متعددة المنصات بكود مصدري وجيد عالي الأداء.
sqflite (SQLite)	lib/services/database_helper.dart	قاعدة بيانات محلية علائقية لتخزين سجلات المشتريات والعمل بدون إنترنت.
ChangeNotifier	PurchasesService, AuthService	نمط إدارة الحالة (State Management) لإشعار الواجهات بالتحديث فوراً.
ListenableBuilder	lib/screens/home_screen.dart	الاستماع لتغييرات الخدمة وإعادة بناء الواجهة الجزئية بأقصى كفاءة.
shared_preferences	lib/services/auth_service.dart	تخزين أزواج المفاتيح والقيم الخفيفة لإدارة الجلسة والنسخ الاحتياطية.
image_picker	lib/widgets/purchase_form_dialog.dart	الوصول للعتاد المادي للهاتف (الكاميرا والمعرض) لالتقاط صور الفواتير.
intl	home_screen.dart, inventory_table.dart	تنسيق الأرقام والعملات المحاسبية بالريال السعودي (ر.س) باللغة العربية.
Base64 Encoding	purchase_form_dialog.dart	تحويل بيانات صور الفواتير لنصوص Data URI مدمجة لتسهيل نقلها وساحتها.

13. الأسئلة الأكاديمية المتوقعة في مناقشة الدكتور وإجاباتها

تم صياغة هذه الأسئلة النموذجية بناءً على محاضرات المنهج الجامعي والأسئلة الشائعة التي يطرحها دكاترة المادة لاختبار مدى فهم الطالب العميق لبناء مشروعه:

1. ما الفرق الجوهرى بين StatelessWidget و StatefulWidget؟ وأين استخدمتم كلا منهما؟

الإجابة النموذجية: الـ `StatelessWidget` هو عنصر واجهة ثابت لا تتغير حالته أو مظهره بعد بنائه، واستخدمناه في `StatCard` و `AppImageview` لأنها عناصر تتلقى بيانات عبر المشيد وتعرضها فقط. أما الـ `StatefulWidget` فهو يمتلك كائن حالة `State` يمكنه إعادة بناء الشاشة عند حدوث تفاعل عبر `setState()`، واستخدمناه في `HomeScreen` و `PurchaseFormDialog` لتحديث الحسابات والقوائم فوراً.

2. لماذا استخدمتم Container في بعض الأماكن و Card في أماكن أخرى؟

الإجابة النموذجية: الـ `Card` هو Widget مخصص يمتلك سمات افتراضية مسبقة للظل `elevation` والحواف وفق تصميم Material Design. بينما الـ `Container` يوفر مرونة مطلقة عبر خاصية `decoration: BoxDecoration` حيث يتيح لنا التحكم المجهرى في لون الخلفية، الحدود `border`، نصف قطر الانحناء `borderRadius`، والظلال المخصصة `boxShadow`، ولهذا اعتمدنا على `Container` لبناء بطاقات الإحصاء لتعكس هوية شركة أثر الدقيقة.

3. ما الفرق بين Row و Column؟ وما المشكلة الشائعة عند استخدام نصوص طويلة داخلهما؟

الإجابة النموذجية: الـ `Row` يرتب الأبناء أفقياً بجانب بعضهم، والـ `Column` يرتبهم رأسياً فوق بعضهم. والمشكلة الشائعة هي حدوث تجاوز لحدود الشاشة (`Overflow / Yellow & Black Bars`) عندما يكون المحتوى أكبر من مساحة الشاشة. وقد قمنا بحل هذه المشكلة بتغليف العناصر النصية بـ `Expanded` أو استخدام `SingleChildScrollView` ليصبح المحتوى قابلاً للتمرير بأمان.

4. لماذا استخدمتم CustomScrollView و SliverList بدلاً من ListView العادى في HomeScreen؟

الإجابة النموذجية: لأن الشاشة الرئيسية تحتوي على عناصر رأسية متنوعة: بطاقات إحصاء، حقل بحث، وسوم فلترة زمنية، ثم قائمة المشتريات. استخدام `ListView` عادى مع عناصر أخرى كان سيَجبرنا على استخدام `shrinkWrap: true` مما يهدر الذاكرة. بينما `CustomScrollView` مع `SliverToBoxAdapter` و `SliverList` يدمج كل مكونات الصفحة في شريط تمرير فيزيائى واحد سلس عالي الكفاءة.

5. كيف قمتم بتطبيق نمط الـ Singleton في كلاس DatabaseHelper؟ وما فائدته؟

الإجابة النموذجية: قمنا بإغلاق المشيد الافتراضى عبر مشيد خاص `DatabaseHelper._privateConstructor()`، وتوفير كائن وحيد ثابت `static final DatabaseHelper instance`. فائدته منع فتح اتصالات متعددة بقاعدة بيانات SQLite مما يمنع قفل القاعدة ويقلل استهلاك الذاكرة العشوائية RAM.

6. كيف يتم حساب إجمالي الفاتورة تلقائياً قبل أن يضغط المستخدم على زر الحفظ؟

الإجابة النموذجية: في كلاس `PurchaseFormDialog`، قمنا بإضافة مستمع `addListener` لمتحكمي النصوص `_quantityController` و `_priceController`، فعند كتابة أي رقم يتم استدعاء `_onCalculatedChanged()` التي تعيد حساب `quantity * unitPrice` فوراً وتحديث الشاشة عبر `setState()`.

7. كيف يعمل البحث اللحظى في قائمة المشتريات وجدول المخزون؟

الإجابة النموذجية: عبر خاصية `onChanged` في حقل `TextField` ، والتي ترسل النص المكتوب إلى `PurchasesService.setSearchQuery()` ، حيث يتم تصفية القائمة الأصلية باستخدام دالة التصفية في الدارت `where()` مع تحويل النصوص لأحرف صغيرة `toLowerCase()` لتتطابق اسم الصنف أو الملاحظات دون الحاجة لإعادة الاستعلام من قاعدة البيانات في كل حرف.

8. كيف يتم تخزين صورة الفاتورة في SQLite؟ وماذا لو قام المستخدم بتغيير هاتفه؟

الإجابة النموذجية: يتم تحويل بايتات الصورة الملتقطة بالكاميرا إلى سلسلة نصية مشفرة بتنسيق `Base64 Data URI` وتخزينها في عمود `invoice_uri` ك نص `TEXT` في SQLite. وبفضل ذلك، عند مزامنة النسخة السحابية لـ Google Drive، يتم رفع الصورة بداخل نص JSON كحزمة واحدة مستقلة، فيستطيع المستخدم استرجاع الفواتير بكامل صورها على أي هاتف جديد فور تسجيل الدخول.

9. ما وظيفة مكتبة flutter_localizations وكيف جعلتم التطبيق يدعم اللغة العربية RTL؟

الإجابة النموذجية: تم تعريف `GlobalMaterialLocalizations` و `GlobalWidgetsLocalizations` داخل `MaterialApp` مع تحديد `locale: Locale('ar', 'SA')` ، مما يجعل جميع الأدوات ومحاذاة النصوص وأدوات التقويم وحركات الانتقال تبدأ تلقائياً من اليمين إلى اليسار (Right-to-Left).

10. كيف يتم منع الشاشات السابقة من البقاء في الذاكرة عند الانتقال من SplashScreen؟

الإجابة النموذجية: عبر استخدام `Navigator.of(context).pushReplacement()` بدلاً من `push()` العادية، مما يحذف `SplashScreen` من مكدس التنقل (Navigation Stack) تماماً، وعندما يضغط المستخدم على زر الرجوع يخرج من التطبيق بدلاً من العودة لشاشة البداية.

14. معجم المصطلحات البرمجية والخاتمة الأكاديمية

14.1 معجم المصطلحات الفنية (Technical Glossary):

- **Widget**: اللبنة الأساسية لبناء كل عنصر مرئي أو غير مرئي في واجهات فلاتر.
- **State Management**: منظومة إدارة وتدفق البيانات وتحديث واجهة المستخدم استجابةً للأحداث.
- **ChangeNotifier**: كلاس في فلاتر يُطلق إشعارات `notifyListeners()` لإعلام الواجهات بوجود بيانات جديدة.
- **ListenableBuilder**: أداة تعيد بناء الجزء المحدد من الشاشة فقط عند ورود إشعار من كائن المراقبة دون هدر الموارد.
- **CRUD**: العمليات الأربع الأساسية في قواعد البيانات: الإضافة (Create)، القراءة (Read)، التعديل (Update)، والحذف (Delete).
- **Singleton Pattern**: نمط تصميم يضمن وجود نسخة واحدة فقط من الكلاس في الذاكرة طوال دورة حياة التطبيق.
- **Base64**: نظام ترميز يحول الملفات الثنائية (كالصور) إلى سلاسل نصية ليسهل تخزينها في قواعد البيانات ونقلها عبر الشبكة.
- **Offline-First**: معمارية برمجية تجعل التطبيق يعمل محلياً بكفاءة 100% دون اشتراط وجود اتصال بالإنترنت.

14.2 قائمة الملفات التي تم فحصها وتوثيقها بدقة:

1. `flutter_app/pubspec.yaml` — إعدادات الحزم والأصول.
2. `flutter_app/android/app/src/main/AndroidManifest.xml` — الصلاحيات وتهيئة الأندرويد.
3. `flutter_app/lib/main.dart` — نقطة التشغيل الرئيسية وإعدادات السمة والتعريب.
4. `flutter_app/lib/models/purchase.dart` — نموذج بيانات الشراء وطرق التحويل `fromJson` و `toJson`.
5. `flutter_app/lib/services/database_helper.dart` — محرك قاعدة بيانات SQLite بنمط Singleton.
6. `flutter_app/lib/services/purchases_service.dart` — إدارة قائمة المشتريات والفاترة والحسابات المالية.
7. `flutter_app/lib/services/auth_service.dart` — إدارة جلسة المستخدم ومصادقة Google ودرائف.
8. `flutter_app/lib/services/backup_sync_service.dart` — خدمات النسخ السحابي وتوليد واسترجاع ملفات .athar.
9. `flutter_app/lib/services/app_update_service.dart` — إدارة التحديثات الآمنة دون المساس بقاعدة البيانات.
10. `flutter_app/lib/theme/app_theme.dart` — ألوان وسمات شركة أثر الفاخرة Material 3.
11. `flutter_app/lib/screens/splash_screen.dart` — شاشة البداية والحركة الانسيابية.
12. `flutter_app/lib/screens/login_screen.dart` — شاشة المصادقة وحماية البيانات.
13. `flutter_app/lib/screens/home_screen.dart` — الشاشة المركزية وسجل المشتريات والإحصائيات.

14. `flutter_app/lib/screens/inventory_table_screen.dart` — جدول المخزون والفرز والنسخ

السريع.

15. `flutter_app/lib/widgets/stat_card.dart` — بطاقة الإحصائيات المستقلة المزودة بظلال وتأثير

فاخر.

16. `flutter_app/lib/widgets/purchase_card.dart` — بطاقة عرض الصنف في السجل وخيارات التعديل

والحذف.

17. `flutter_app/lib/widgets/purchase_form_dialog.dart` — نموذج تسجيل المشتريات واحتساب

التكلفة وتصوير الفاتورة.

18. `flutter_app/lib/widgets/invoice_viewer_dialog.dart` — عارض الفواتير المكرر.

19. `flutter_app/lib/widgets/app_image_view.dart` — عارض الصور الذكي للبيانات والشبكة

والملفات.

14.3 الخاتمة الأكاديمية:

بحمد الله وتوفيقه، تم تصميم وبرمجة وتوثيق مشروع "إدارة مخزون شركة أثر" بأعلى درجات الاحترافية البرمجية، مجسداً المعايير الأكاديمية الصارمة المقررة في المنهج الجامعي. يثبت هذا العمل قدرة إطار العمل **Flutter** على تقديم حلول محاسبية وإدارية متكاملة وفائقة السرعة للأعمال والشركات، تجمع بين جمال المظهر الخارجي، وسرعة محركات قواعد البيانات المحلية SQLite، وأمان المزامنة السحابية بحسابات Google Drive.

تم بحمد الله وتوفيقه | مشروع إدارة مخزون شركة أثر 2025 - 2026م